

Disciplina: DCA 3703 - PROGRAMAÇÃO PARALELA

Professor: Samuel Xavier De Souza

Nome: José Felix Rodrigues Anselmo

Tarefa 1

1 - Enunciado

Implemente um programa em C que calcule uma aproximação de π usando uma série matemática, variando o número de iterações e medindo o tempo de execução. Compare os valores obtidos com o valor real de π e analise como a acurácia melhora com mais processamento. Reflita sobre como esse comportamento se repete em aplicações reais que demandam resultados cada vez mais precisos, como simulações físicas e inteligência artificial.

2 - Metodologia

Para gerar a aproximação do número pi foi usada a série de Leibniz que é escrita da seguinte forma:

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \dots = \sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1}$$

figura 1: Série de leibniz

Após entender como funciona a aproximação, foi desenvolvido o código em C que executaria as aproximações, com vários valores limites de iterações foi usado ainda o walltime com o parâmetro clock_gettime para calcular o tempo de execução desse código.

Além disso, foi usado o erro absoluto para calcular a diferença entre o valor obtido e o valor real.

2.2 Algoritmo

Etapas do algoritmo:

1. Inicializar a variável pi com o valor 0.0.
2. Percorrer as iterações:
Para cada iteração k, de 0 até N-1:
 - Calcular o termo: $1 / 2^{k+1}$
 - Aplicar o sinal alternado:
 - Se k for par, o termo será positivo;
 - Se k for ímpar, o termo será negativo.
 - Adicionar o termo à variável acumuladora pi.
3. Obter a aproximação de π : $4 \times \pi_{\text{aprox}} = \pi$
4. Calcular o erro absoluto: $\text{Erro} = | \pi_{\text{aprox}} - \pi_{\text{real}} |$

2.3 - Ambiente de Execução

Componente	Especificação
Notebook	Lenovo IdeaPad 3 15ALC6
SO	Ubuntu 26.04 LTS
Kernel	Linux 7.0.0-29-generic
CPU	AMD Ryzen 5 5500U — 6C/12T
Arquitetura	x86_64
RAM	9,05 GiB
Desktop	GNOME 50.1
Shell	Zsh 5.9
Terminal	Kitty 0.45.0
Compilador	GCC

Tabela 1 - Configurações da máquina de teste

3 - Código Fonte

O código será executado de forma sequencial (serial), iterativa e em uma única thread. Ele calcula a aproximação de π pela Série de Leibniz, testando diferentes quantidades de iterações e medindo o tempo de cada execução.

```
#include <stdio.h>

#include <math.h>

#include <time.h>

#define _USE_MATH_DEFINES

long double calcularPiLeibniz(int interacao) {

    long double pi = 0.0L;

    long double termo;

    for (int i = 0; i < interacao; i++) {

        if (i % 2 == 0) {

            termo = 1.0 / (2 * i + 1);

        } else {

            termo = -1.0 / (2 * i + 1);

        }

        pi = pi + termo;

    }

    return pi * 4.0L;

}

int main(void) {

    struct timespec inicioIteracao, fimIteracao;

    struct timespec inicioTotal, fimTotal;
```

```

const double pi_real = M_PI;

int listaDelteracoes[] = {10, 100, 1000, 10000, 100000, 1000000, 10000000, 100000000,
1000000000};

int n = sizeof(listaDelteracoes) / sizeof(listaDelteracoes[0]);
int j;

printf("%-15s %-22s %-18s %-15s\n", "Iteracoes", "Pi aproximado", "Erro absoluto", "Walltime
(s)");
printf("-----\n");

clock_gettime(CLOCK_MONOTONIC, &inicioTotal);

for (j = 0; j < n; j++) {
    int interacao = listaDelteracoes[j];

    clock_gettime(CLOCK_MONOTONIC, &iniciolteracao);

    long double piCalculado = calcularPiLeibniz(interacao);

    clock_gettime(CLOCK_MONOTONIC, &fimlteracao);

    long double erro = fabs(pi_real - piCalculado);

    double tempolteracao = (fimlteracao.tv_sec - iniciolteracao.tv_sec) + (fimlteracao.tv_nsec -
iniciolteracao.tv_nsec) / 1e9;

    printf("%-15d %-22.15Lf %-18.15Lf %-15.6f\n", interacao, piCalculado, erro, tempolteracao);
}

clock_gettime(CLOCK_MONOTONIC, &fimTotal);

double tempoTotal = (fimTotal.tv_sec - inicioTotal.tv_sec) + (fimTotal.tv_nsec -
inicioTotal.tv_nsec) / 1e9;

```

```

printf("-----\n");
printf("Tempo total (walltime) para todas as iteracoes: %.6f s\n", tempoTotal);

return 0;
}

```

4 - Resultados e Conclusões

A imagem abaixo mostra a saída após a execução do código, para critério de comparação foi usado valor de pi $\pi \approx 3.14159265358979323846$ através da biblioteca <math.h>.

```

) time ./main
Iteracoes      Pi aproximado      Erro absoluto      Walltime (s)
-----
10              3.041839618929402  0.099753034660391  0.000002
100             3.131592903558553  0.009999750031240  0.000004
1000            3.140592653839793  0.000999997500000  0.000027
10000           3.141492653590043  0.000099999999750  0.000257
100000          3.141582653589794  0.000009999999999  0.002143
1000000         3.141591653589793  0.000000100000000  0.014486
10000000        3.141592553589793  0.000000010000000  0.057983
100000000       3.141592643589794  0.000000009999999  0.568951
1000000000      3.141592652589795  0.000000000999998  5.683199
-----
Tempo total (walltime) para todas as iteracoes: 6.327231 s
./main 6,33s user 0,00s system 99% cpu 6,332 total

```

figura 2: Resultado da execução

Observando na imagem acima, podemos destacar que quanto mais iterações o programa faz, mais o valor aproximado de pi melhora e menor se torna o erro absoluto. O tempo de execução cresce à medida que aumenta a quantidade de iterações. Os resultados obtidos se assemelham muito ao que acontece no treinamento de modelos de inteligência artificial, no qual se aplicam cálculos complexos para aumentar a precisão o resultado, o mesmo acontece em simulações físicas que tentam aproximar ao máximo do contexto real inserindo variáveis que representam atividades da natureza.